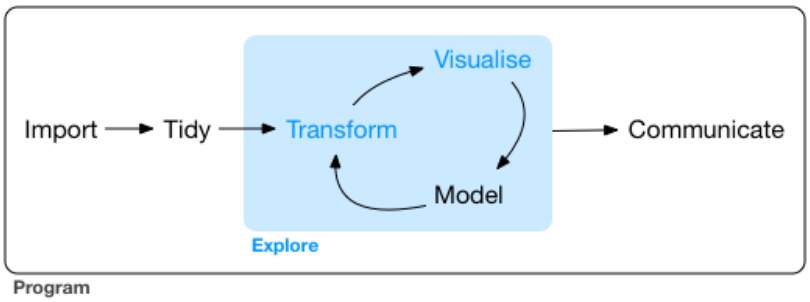
CUNEF
UNIVERSIDAD

Transformation

Data Analysis & Information Extraction

Table of contents

Transformation is one of the key tasks for the correct use of the data, but is very rare to find the data fully ready for an analysis, so we probabluy need to do some tasks to adapt it for the analysis we want to do.



Generally we will need...

- Create new variables
- Sum up some of them.
- Rename variables
- Reorder
- Grouping

... among other things.

Transformation: dplyr

We will use an special package designed inside the *tidyverse* universe for this purpose... **dplyr**

This package has his own language (verbs) for doing the different tasks of the transformation process...

- Working with rows...
  - **arrange** to order rows
  - **slice** to subset rows
  - **filter** to select rows according to a certain condition
- Working with columns...
  - **select** to choose columns
  - **rename** to rename columns
  - **mutate** to create new columns
- And other much more...

Basic *dplyr* rules

All verbs work similar:

- first argument is the data frame
- second and subsequent arguments are used to specify what to do with the data frame
- each verb will return a new data frame modified

All verbs can be **concatenated** using the pipe `%>%`

For starting working with **dplyr** we will need to load it...

```
library(tidyverse)  # Do library(dplyr) for the package only
```

Basic Manipulation

We are going to use the `dplyr` library that is contained inside `tidyverse`. This library provide us advance functions to manipulate the data substituting the majority of functionalities we have in R base.

```
library(tidyverse)
library(gapminder)
```

If you are working with an older version of R maybe you need to load the library directly using `library(dplyr)`.

Selecting columns: `select`

We can use `select` verb to choose which columns we want to work with.

```
gapminder %>%
  select(country, year, gdpPercap)
```

```
# A tibble: 1,704 x 3
   country      year gdpPercap
   <fct>       <int>     <dbl>
 1 Afghanistan  1952      779.
 2 Afghanistan  1957      821.
 3 Afghanistan  1962      853.
 4 Afghanistan  1967      836.
 5 Afghanistan  1972      740.
 6 Afghanistan  1977      786.
 7 Afghanistan  1982      978.
 8 Afghanistan  1987      852.
 9 Afghanistan  1992      649.
10 Afghanistan  1997      635.
# i 1,694 more rows
```

Look that we have use the **pipe** operator included into de tidyverse package. This allow us to concatenate operations simplifying manipulation. Pipes pass the output of last operation as input for the first argument of next operation.

```
# Without pipe
select(gapminder, country, year, gdpPercap)
```

```
# A tibble: 1,704 x 3
   country      year gdpPercap
   <fct>       <int>     <dbl>
 1 Afghanistan  1952      779.
 2 Afghanistan  1957      821.
 3 Afghanistan  1962      853.
 4 Afghanistan  1967      836.
 5 Afghanistan  1972      740.
 6 Afghanistan  1977      786.
 7 Afghanistan  1982      978.
 8 Afghanistan  1987      852.
 9 Afghanistan  1992      649.
10 Afghanistan  1997      635.
# i 1,694 more rows
```

```
# With pipe
gapminder %>%
  select(country, year, gdpPercap)
```


```
# A tibble: 1,704 x 3
   country      year gdpPercap
   <fct>       <int>     <dbl>
 1 Afghanistan  1952      779.
 2 Afghanistan  1957      821.
 3 Afghanistan  1962      853.
 4 Afghanistan  1967      836.
 5 Afghanistan  1972      740.
 6 Afghanistan  1977      786.
 7 Afghanistan  1982      978.
 8 Afghanistan  1987      852.
 9 Afghanistan  1992      649.
10 Afghanistan  1997      635.
# i 1,694 more rows
```

More about the pipe `%>%`

Pipe is a technique designed to pass data from a process to another.
The structure has been implemented in the **magrittr** package (that is included inside **tidyverse**)

```
gapminder %>%
  select(country, year, lifeExp)
```

1. Line 1: We initially start with the whole data set
2. Line 2: Then we use select to have a subset of three columns (country, year, lifeExp)

Combined with **ggplot** we can pass the resulted processing with dplyr to the plot we want to create

```
gapminder %>%
  filter(year == 2007) %>%
  ggplot(., aes(x = lifeExp, fill = continent)) +
  geom_histogram(bins = 100)
```

Pipe operations are very useful and R base has incorporated this kind of operations too. In 2021 with the release of R 4.1.0 an R base pipe operator (`|>`) was introduced making pipe operations always available.
Both pipes work similarly in simple case but have some differences. We are going to work with the *tidyverse* one but take this into account when reading books, code examples that can use the other pipe.

Be aware that any operation applied needs to be saved, into a new variable if we want to keep it.

```
gapminder_new <- gapminder %>%
  select(country, year, gdpPercap)
```

Selection options

You can select a **range of columns**, for example all columns from `country` to `lifeExp`:

```
gapminder %>%
  select(country:lifeExp)
```

```
# A tibble: 1,704 x 4
   country     continent  year lifeExp
   <fct>       <fct>     <int>   <dbl>
```

```
 1 Afghanistan Asia       1952    28.8
 2 Afghanistan Asia       1957    30.3
 3 Afghanistan Asia       1962    32.0
 4 Afghanistan Asia       1967    34.0
 5 Afghanistan Asia       1972    36.1
 6 Afghanistan Asia       1977    38.4
 7 Afghanistan Asia       1982    39.9
 8 Afghanistan Asia       1987    40.8
 9 Afghanistan Asia       1992    41.7
10 Afghanistan Asia       1997    41.8
# i 1,694 more rows
```

You can select all columns excep some of them using - syntax:

```
gapminder %>%
  select(-continent,-pop,-gdpPercap)
```

```
# A tibble: 1,704 x 3
   country      year lifeExp
   <fct>       <int>   <dbl>
 1 Afghanistan  1952    28.8
 2 Afghanistan  1957    30.3
 3 Afghanistan  1962    32.0
 4 Afghanistan  1967    34.0
 5 Afghanistan  1972    36.1
 6 Afghanistan  1977    38.4
 7 Afghanistan  1982    39.9
 8 Afghanistan  1987    40.8
 9 Afghanistan  1992    41.7
10 Afghanistan  1997    41.8
# i 1,694 more rows
```

You can combine the last two options to exclude a range of columns:

```
gapminder %>%
  select(-(lifeExp:country))
```

```
# A tibble: 1,704 x 2
        pop gdpPercap
      <int>     <dbl>
 1  8425333      779.
```

```
 2  9240934      821.
 3 10267083      853.
 4 11537966      836.
 5 13079460      740.
 6 14880372      786.
 7 12881816      978.
 8 13867957      852.
 9 16317921      649.
10 22227415      635.
# i 1,694 more rows
```

You can use advanced select using **tidy select** functions:

```
gapminder %>%
  select(starts_with("co"))
```

```
# A tibble: 1,704 x 2
   country     continent
   <fct>       <fct>
 1 Afghanistan Asia
 2 Afghanistan Asia
 3 Afghanistan Asia
 4 Afghanistan Asia
 5 Afghanistan Asia
 6 Afghanistan Asia
 7 Afghanistan Asia
 8 Afghanistan Asia
 9 Afghanistan Asia
10 Afghanistan Asia
# i 1,694 more rows
```

There are different functions you can use for advanced selection:

- `starts_with("abc")` will select all columns that start with `abc`
- `ends_with("abc")` will select all columns that end with `abc`
- `contains("abc")` column that contains `abc`
- `matches("(.)//1")` use regular expression to select the column
- `num_range("x",1:3)` will select x1, x2 and x3

Finally, there is an option were you can select `everything`. Useful to put some columns first and then the rest...

```
gapminder %>%
  select(year, everything())
```

```
# A tibble: 1,704 x 6
    year country     continent lifeExp      pop gdpPercap
   <int> <fct>       <fct>       <dbl>    <int>     <dbl>
 1  1952 Afghanistan Asia         28.8  8425333      779.
 2  1957 Afghanistan Asia         30.3  9240934      821.
 3  1962 Afghanistan Asia         32.0 10267083      853.
 4  1967 Afghanistan Asia         34.0 11537966      836.
 5  1972 Afghanistan Asia         36.1 13079460      740.
 6  1977 Afghanistan Asia         38.4 14880372      786.
 7  1982 Afghanistan Asia         39.9 12881816      978.
 8  1987 Afghanistan Asia         40.8 13867957      852.
 9  1992 Afghanistan Asia         41.7 16317921      649.
10  1997 Afghanistan Asia         41.8 22227415      635.
# i 1,694 more rows
```

One useful function with `select` is that you can change the name of the columns on selection. Look at the example...

```
gapminder %>%
  select( state = country )
```

```
# A tibble: 1,704 x 1
   state
   <fct>
 1 Afghanistan
 2 Afghanistan
 3 Afghanistan
 4 Afghanistan
 5 Afghanistan
 6 Afghanistan
 7 Afghanistan
 8 Afghanistan
 9 Afghanistan
10 Afghanistan
# i 1,694 more rows
```

Sum up: `select`

- `select()` chooses columns on the data frame and returns a new data frame with the selection.
- There are several ways to select columns adding names directly, range or with advance selection functions. You can select all and remove columns too.
- You can change the order writing the first the columns you want at the begining.
- You can use it to change the name of the column on selection using `newname = column` syntax.

Sorting rows: arrange

You can sort the rows of the data frame using `arrange` verb.

We want to have the data per country and year sorted from lower to high life expectancy:

```
gapminder %>%
  select(country, year, lifeExp) %>%
  arrange(lifeExp)
```

```
# A tibble: 1,704 x 3
   country       year lifeExp
   <fct>        <int>   <dbl>
 1 Rwanda        1992    23.6
 2 Afghanistan   1952    28.8
 3 Gambia        1952    30
 4 Angola        1952    30.0
 5 Sierra Leone  1952    30.3
 6 Afghanistan   1957    30.3
 7 Cambodia      1977    31.2
 8 Mozambique    1952    31.3
 9 Sierra Leone  1957    31.6
10 Burkina Faso  1952    32.0
# i 1,694 more rows
```

As you can see the data is now sorted from lower to high `lifeExp`, we can do the opposite, sorting from high to low. Just adding `desc()`

```
gapminder %>%
  select(country, year, lifeExp) %>%
  arrange(desc(lifeExp))
```

```
# A tibble: 1,704 x 3
   country           year lifeExp
   <fct>            <int>   <dbl>
 1 Japan             2007    82.6
 2 Hong Kong, China  2007    82.2
 3 Japan             2002    82
 4 Iceland           2007    81.8
 5 Switzerland       2007    81.7
 6 Hong Kong, China  2002    81.5
 7 Australia         2007    81.2
 8 Spain             2007    80.9
 9 Sweden            2007    80.9
10 Israel            2007    80.7
# i 1,694 more rows
```

You can sort the data using multiple columns. In these cases the data will be sorted first using first column, then second in case of draw between rows. Look at the following code how the data has been sorted?

```
gapminder %>%
  select(country, year, lifeExp) %>%
  arrange(year,desc(lifeExp))
```

```
# A tibble: 1,704 x 3
   country         year lifeExp
   <fct>          <int>   <dbl>
 1 Norway          1952    72.7
 2 Iceland         1952    72.5
 3 Netherlands     1952    72.1
 4 Sweden          1952    71.9
 5 Denmark         1952    70.8
 6 Switzerland     1952    69.6
 7 New Zealand     1952    69.4
 8 United Kingdom  1952    69.2
 9 Australia       1952    69.1
10 Canada          1952    68.8
# i 1,694 more rows
```

In case of having NA values this are always sorted at the end of the data frame**

Exercise

Sort the gapminder data per country, what has happen how it is sorted?
Hint: The characters in computing are numbers internally, and they have a ordinal configuration se we can sort them properly.

Sum up: arrange

- `arrange()` orders in ascending order by default (numbers)
- In the case of strings, `arrange()` will order using the alphabetical order.
- We can use `desc()` function to change the order.
- If more than one variable is provided, the first will be used for doing the order, in case of draws the next variables will be used for deciding the order.
- Missing values will be order at the end.

Creating columns: `mutate`

You can create new columns using `mutate` verb to add more information in your data frame.

For example, we want to add a new column called `gdp` that will be the result of multiplying the population (pop) by `gpdPercap`.

```
gapminder %>%
  mutate(
    gdp = pop * gdpPercap
  ) %>%
  select(pop, gdpPercap, gdp)
```

```
# A tibble: 1,704 x 3
       pop gdpPercap          gdp
     <int>     <dbl>        <dbl>
 1  8425333      779.  6567086330.
 2  9240934      821.  7585448670.
 3 10267083      853.  8758855797.
 4 11537966      836.  9648014150.
 5 13079460      740.  9678553274.
 6 14880372      786. 11697659231.
 7 12881816      978. 12598563401.
 8 13867957      852. 11820990309.
 9 16317921      649. 10595901589.
10 22227415      635. 14121995875.
```

```
# i 1,694 more rows
```

Some important note to consider with `mutate`:

- You can create more than one variable in a single call to the function. Each variable will be in a different argument.
- You can re-use a variable (it will be overwritten).

For creating columns we have some useful functions we can use:

- Arithmetic operators (+, -, *, /)
- Modular arithmetic operators: integer division `%/%` and module `%%`
- Logarithm
- Cumulative aggregations: `cumsum()`, `cumprod()`, `cummin()` and `cummax()`
- Logical comparisons
- Rankings: `min_rank()`

```
x = c(1,3,2,7,6)
# Ranking
min_rank(x)
```

```
[1] 1 3 2 5 4
```

```
# Module
33 %% 10
```

```
[1] 3
```

```
# Cumulative sum
cumsum(x)
```

```
[1]  1  4  6 13 19
```

Exercise

In `gapminder` data frame that we provide you in the code below create the perform the following operations, all operations must be done in the same structure and verb using pipes.

1. Create a new column `total_pop` that will be the `sum` of all `pop`. This is the total population.
2. Create a new column `pop_rate` that will be the division of `pop` per `total_pop`. Population % among total population.
3. Sort the data frame per `pop_rate` from higher to lower population rate.
4. The final output should be a data frame with the columns `country`, `pop`, `pop_rate`.

Sum up: `mutate`

- `mutate()` creates new columns or modify existing columns.
- You can create more than one column in the same call and reuse a newly created column to create another one.
- There are different functions and operators we can use to compute complex logics and create new columns.

Other manipulations

Selecting rows: `slice`

You can select certain rows based on the index:

```
gapminder %>%
  select(country, year, lifeExp) %>%
  slice(1:3)
```

```
# A tibble: 3 x 3
  country      year lifeExp
  <fct>       <int>   <dbl>
1 Afghanistan  1952    28.8
2 Afghanistan  1957    30.3
3 Afghanistan  1962    32.0
```

We have selected the first three rows of the data frame.

> Exercise
>
> Add a new operation to the output so the sorted by `pop_rate` returns only the top 5 countries per highest rate of population.

Selecting rows: `head`

Works similar to `slice`, but it always start from the top. So we only need to provide the number of rows we want to have from the top of the data frame.

```
gapminder %>%
  select(country, year, lifeExp) %>%
  head(3)
```

```
# A tibble: 3 x 3
  country      year lifeExp
  <fct>       <int>   <dbl>
1 Afghanistan  1952    28.8
2 Afghanistan  1957    30.3
3 Afghanistan  1962    32.0
```

Unique rows or values: `distinct`

Sometimes we will need to know how many different categories or values we have, similarly to `unique` we have `distinct`.

```
gapminder %>%
  distinct(continent)
```

```
# A tibble: 5 x 1
  continent
  <fct>
1 Asia
2 Europe
3 Africa
4 Americas
5 Oceania
```

The relevant feature with `distinct` is that we can use it for the two or more columns so we can know combinations of values.

```
gapminder %>%
  distinct(continent, year)
```

```
# A tibble: 60 x 2
   continent  year
   <fct>     <int>
 1 Asia       1952
 2 Asia       1957
 3 Asia       1962
 4 Asia       1967
 5 Asia       1972
 6 Asia       1977
 7 Asia       1982
 8 Asia       1987
 9 Asia       1992
10 Asia       1997
# i 50 more rows
```

Filtering

```
library(tidyverse)
library(gapminder)
```

In R base we have learnt how to filter our data frames to have only the rows that match certain criteria. With `dplyr` we can filter too our data frames.

We want only the rows whose `lifeExp` is higher than 80 years.

```
gapminder %>%
  filter(lifeExp > 80)
```

```
# A tibble: 21 x 6
   country          continent  year lifeExp      pop gdpPercap
   <fct>            <fct>     <int>   <dbl>    <int>     <dbl>
 1 Australia        Oceania    2002    80.4 19546792    30688.
 2 Australia        Oceania    2007    81.2 20434176    34435.
 3 Canada           Americas   2007    80.7 33390141    36319.
 4 France           Europe     2007    80.7 61083916    30470.
 5 Hong Kong, China Asia       2002    81.5  6762476    30209.
 6 Hong Kong, China Asia       2007    82.2  6980412    39725.
 7 Iceland          Europe     2002    80.5   288030    31163.
 8 Iceland          Europe     2007    81.8   301931    36181.
 9 Israel           Asia       2007    80.7  6426679    25523.
10 Italy            Europe     2002    80.2 57926999    27968.
# i 11 more rows
```

Remember that the filter we have applied doesn't modify the data frame, we will need to store the result in a new one if we want to use the result in the future.

```
gapminder_h80 <- gapminder %>%
  filter(lifeExp > 80)
```

We can build much more complex conditions depending on our needs, remember all filtering possibilities:

operator	definition	...	operator	definition
<	Less than		x & y	x AND y
<=	Less than or equal		x | y	x OR y
>	More		is.na(x)	is x a NA value
>=	More than or equal		x %in% y	x in vector y
==	Equal		!x	opposite of condition x
!=	Distinct			

For example we want to filter the rows whose `lifeExp` is higher than 80 years and for "Spain"

```
gapminder %>%
  filter(lifeExp > 80, country == "Spain")
```

```
# A tibble: 1 x 6
  country continent  year lifeExp      pop gdpPercap
  <fct>   <fct>     <int>   <dbl>    <int>     <dbl>
1 Spain   Europe     2007    80.9 40448191    28821.
```

In the example above, we have used , as in filter verb is similar to use operator `&`. For example the following filter is equivalent to this one.

```
gapminder %>%
  filter(lifeExp > 80 & country == "Spain")
```

```
# A tibble: 1 x 6
  country continent  year lifeExp      pop gdpPercap
  <fct>   <fct>     <int>   <dbl>    <int>     <dbl>
1 Spain   Europe     2007    80.9 40448191    28821.
```

Exercise

Now try the following filtering options, one by one to the same `gapminder` data frame.

1. All rows which life expectancy is between 40 and 60 (both included).
2. All rows which life expectancy is between 40 and 60 (both excluded), and for the continent "Asia".
3. All rows which GPD per capita is higher than 100000 or with a lifeExp lower than 40, for countries located in "Europe" and "America".
4. Which are the top 5 distinct countries with the highest pop with a life expectancy higher than 90. Hint: You may need to use basic manipulation as seen in last lesson.

Missing Values on Filters

Missing values (NA) are something inevitable and highly frequently in the data.

All operations done with missing values will return another missing value...

```
NA == 2
```

```
[1] NA
```

```
NA + 3
```

```
[1] NA
```

```
NA == NA
```

```
[1] NA
```

When filtering the resulted data frame will only contain data that makes **TRUE** the condition. Data making **FALSE** the condition and missing values will be excluded.

```
df <- tibble(x = c(1, NA, 3))
filter(df, x > 1)
```

```
# A tibble: 1 x 1
      x
  <dbl>
1     3
```

```r
filter(df, is.na(x) | x > 1)
```

```
# A tibble: 2 x 1
      x
  <dbl>
1    NA
2     3
```

Aggregations

In some case we will need to retrieve summary information of the data frame needing to aggregate the data into several KPIs.

Transforming with `summarise`

For example, we need to retrieve some basic statistics of the whole data frame, this way we can understand the main data frame insights.

For doing that we can use `summarise` verb, with this function we can compute aggregated results for the data frame. We need to provide the column name where the result will be stored and the operation to perform

```
gapminder %>%
  summarise(
    min_lifeExp = min(lifeExp),
    max_lifeExp = max(lifeExp),
    avg_lifeExp = mean(lifeExp)
  )
```

```
# A tibble: 1 x 3
  min_lifeExp max_lifeExp avg_lifeExp
        <dbl>       <dbl>       <dbl>
1        23.6        82.6        59.5
```

Remember to be careful with NAs, if there are NA values present on the observations of the data frame the aggregation operations won't succeed.

```
library(fivethirtyeight)
```

```
Some larger datasets need to be installed separately, like senators and
house_district_forecast. To install these, we recommend you install the
fivethirtyeightdata package by running:
install.packages('fivethirtyeightdata', repos =
'https://fivethirtyeightdata.github.io/drat/', type = 'source')
```

```
biopics %>%
  summarise(
    total_box_office = sum(box_office),
    avg_box_office = mean(box_office)
  )
```

```
# A tibble: 1 x 2
  total_box_office avg_box_office
             <dbl>          <dbl>
1               NA             NA
```

If that is the case remember to include `na.rn = TRUE` so NA values are not considered in the aggregations...

```
biopics %>%
  summarise(
    total_box_office = sum(box_office, na.rm =T),
    avg_box_office = mean(box_office, na.rm = T)
  )
```

```
# A tibble: 1 x 2
  total_box_office avg_box_office
             <dbl>          <dbl>
1      10042773040      22981174.
```

summarise vs mutate

Some cases we have confusion on when to use `summarise` and when to use `mutate` as both verbs create columns but they work differently.

- `mutate` creates new columns, but preserves the original structure of the data frame. It is used for adding new information to the data frame in a new variable.
- `summarise` creates new columns that are the aggregations of the whole or groups of data in the data frame. The final output of the `summarise` changes the structure of the data frame completely providing a summarization of the data based on the requested operations.

You can use aggregation functions with `mutate` but they will work differently as the returned value will be replicated in all observations...

For example, we want to compare the gdpPercap of each country with the world average, assuming that the average of the data frame is the world average how we can due that?

```r
gapminder_67 <- gapminder %>% filter(year == 1967)

gapminder_67 %>%
  mutate(
    world_avg_gdp = mean(gdpPercap), # will compute the mean of the whole data for all registers
    gpd_vs_world = gdpPercap - world_avg_gdp
  ) %>%
  select(country, gdpPercap, world_avg_gdp, gpd_vs_world)
```

```
# A tibble: 142 x 4
   country     gdpPercap world_avg_gdp gpd_vs_world
   <fct>           <dbl>         <dbl>        <dbl>
 1 Afghanistan      836.         5484.       -4647.
 2 Albania         2760.         5484.       -2723.
 3 Algeria         3247.         5484.       -2237.
 4 Angola          5523.         5484.          39.1
 5 Argentina       8053.         5484.        2569.
 6 Australia      14526.         5484.        9042.
 7 Austria        12835.         5484.        7351.
 8 Bahrain        14805.         5484.        9321.
 9 Bangladesh       721.         5484.       -4762.
10 Belgium        13149.         5484.        7665.
# i 132 more rows
```

Using aggregation functions with mutate is helpful for making this kind of analysis and understand relevant deviations in the observations.

Grouping rows with `group_by`

In certain occasions we would like to perform grouping in our data, grouping is useful for several reasons:

- Computing aggregations by groups i.e totals per year
- Creating some columns and apply some *logics* per groups of observations only affecting rows of each group.

For generating groups on the data we should use `group_by` function...

Example: We want to have the same stats but per year in gapminder...

```
gapminder %>%
  group_by(year) %>%
  summarise(
    min_lifeExp = min(lifeExp),
    max_lifeExp = max(lifeExp),
    avg_lifeExp = mean(lifeExp)
  )
```

```
# A tibble: 12 x 4
    year min_lifeExp max_lifeExp avg_lifeExp
   <int>       <dbl>       <dbl>       <dbl>
 1  1952        28.8        72.7        49.1
 2  1957        30.3        73.5        51.5
 3  1962        32.0        73.7        53.6
 4  1967        34.0        74.2        55.7
 5  1972        35.4        74.7        57.6
 6  1977        31.2        76.1        59.6
 7  1982        38.4        77.1        61.5
 8  1987        39.9        78.7        63.2
 9  1992        23.6        79.4        64.2
10  1997        36.1        80.7        65.0
11  2002        39.2        82          65.7
12  2007        39.6        82.6        67.0
```

With the groups now the results are generated for each group created using group by.

You can create groups with two or more variables...

```
gapminder %>%
  group_by(year, continent) %>%
  summarise(
    min_lifeExp = min(lifeExp),
    max_lifeExp = max(lifeExp),
    avg_lifeExp = mean(lifeExp)
  )
```

```
`summarise()` has regrouped the output.
i Summaries were computed grouped by year and continent.
i Output is grouped by year.
i Use `summarise(.groups = "drop_last")` to silence this message.
i Use `summarise(.by = c(year, continent))` for per-operation grouping
  (`?dplyr::dplyr_by`) instead.
```

```
# A tibble: 60 x 5
# Groups:   year [12]
    year continent min_lifeExp max_lifeExp avg_lifeExp
   <int> <fct>           <dbl>       <dbl>       <dbl>
 1  1952 Africa           30          52.7        39.1
 2  1952 Americas         37.6        68.8        53.3
 3  1952 Asia             28.8        65.4        46.3
 4  1952 Europe           43.6        72.7        64.4
 5  1952 Oceania          69.1        69.4        69.3
 6  1957 Africa           31.6        58.1        41.3
 7  1957 Americas         40.7        70.0        56.0
 8  1957 Asia             30.3        67.8        49.3
 9  1957 Europe           48.1        73.5        66.7
10  1957 Oceania          70.3        70.3        70.3
# i 50 more rows
```

Look that when using more than one variable, the groups don't disapear, as R will maintain groups for *variables-1*. In this case we have grouped per year and continent, so when the aggregation finishes R will mantain groups for year but ungroup continent.

If we want to remove groups we can do it with ungroup verb.

```
gapminder %>%
  group_by(year, continent) %>%
  summarise(
    min_lifeExp = min(lifeExp),
    max_lifeExp = max(lifeExp),
    avg_lifeExp = mean(lifeExp)
  ) %>%
  ungroup(year)
```

```
`summarise()` has regrouped the output.
i Summaries were computed grouped by year and continent.
i Output is grouped by year.
i Use `summarise(.groups = "drop_last")` to silence this message.
i Use `summarise(.by = c(year, continent))` for per-operation grouping
  (`?dplyr::dplyr_by`) instead.
```

```
# A tibble: 60 x 5
    year continent min_lifeExp max_lifeExp avg_lifeExp
   <int> <fct>           <dbl>       <dbl>       <dbl>
 1  1952 Africa           30          52.7        39.1
```

```
 2  1952 Americas          37.6        68.8        53.3
 3  1952 Asia              28.8        65.4        46.3
 4  1952 Europe            43.6        72.7        64.4
 5  1952 Oceania           69.1        69.4        69.3
 6  1957 Africa            31.6        58.1        41.3
 7  1957 Americas          40.7        70.0        56.0
 8  1957 Asia              30.3        67.8        49.3
 9  1957 Europe            48.1        73.5        66.7
10  1957 Oceania           70.3        70.3        70.3
# i 50 more rows
```

Look that we only applied `ungroup` to year variable, as continent groups are removed by default when applying `summarise` function. Summarise will remove the first level of the grouped variables always.

Using basic verbs with groups

Grouping is not only used for aggregations you can create groups and perform the same basic manipulation instructions but this time them will be applied considering the existing groups.

For example, we want to have the data frame ordered so for each year we have the countries ordered from lower to higher `gdpPercap`

```
gapminder_sorted <- gapminder %>%
  group_by(year) %>% # now all observations are grouped for their year
  arrange(gdpPercap) # data frame will be sorted by lower to higher gdp but for each year

gapminder_sorted %>%
  filter(year == 1977) %>%
  select(year, country, gdpPercap) %>%
  slice(1:10)
```

```
# A tibble: 10 x 3
# Groups:   year [1]
    year country    gdpPercap
   <int> <fct>          <dbl>
 1  1977 Myanmar         371
 2  1977 Mozambique      502.
 3  1977 Eritrea         506.
 4  1977 Cambodia        525.
 5  1977 Burundi         556.
 6  1977 Ethiopia        557.
```

```
 7  1977 Liberia          640.
 8  1977 Bangladesh       660.
 9  1977 Malawi           663.
10  1977 Rwanda           670.
```

Build it functions... count

Dplyr contains build in functions that perform quick operations, making simple performing some tasks and not needing to grouping or aggregating.

For example we want to now how many observations we have for each continent...

```
gapminder %>%
  count(continent)
```

```
# A tibble: 5 x 2
  continent     n
  <fct>     <int>
1 Africa      624
2 Americas    300
3 Asia        396
4 Europe      360
5 Oceania      24
```